

Machine Shop Digitizer

Claude Handoff Document — History Page Refactor

This document gives a complete briefing to a fresh Claude session. Read it top to bottom before writing any code. All context, current file states, and exact changes required are included.

1. Project Overview

Next.js 15 App Router + TypeScript application that digitises handwritten manufacturing log sheets using AI (Gemini 2.0 Flash primary, Mistral pixtral-12b fallback). Data is stored in MongoDB via Mongoose.

Layer	Technology
Framework	Next.js 15 (App Router)
Language	TypeScript
UI	Tailwind CSS + shadcn/ui + lucide-react
Charts	Recharts
Database	MongoDB + Mongoose
AI Primary	Google Gemini 2.0 Flash (@google/generative-ai)
AI Fallback	Mistral pixtral-12b-latest (@mistralai/mistralai)

2. Current File Structure

```
machine-shop-app/
├── app/
│   ├── api/
│   │   ├── upload/route.ts      # POST – Gemini/Mistral OCR, save to MongoDB
│   │   ├── documents/route.ts   # GET – list all records (no image field)
│   │   ├── documents/[id]/route.ts # GET, PUT, DELETE – single record
│   │   └── analytics/route.ts   # GET – MongoDB aggregations for dashboard
│   ├── dashboard/page.tsx       # Recharts analytics dashboard
│   ├── history/page.tsx         # History/search table ← BEING REPLACED
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx                 # Upload & Review (split-screen)
├── components/
│   ├── ui/ (button, card, badge, input, label, select, alert)
│   └── Navbar.tsx
├── lib/
│   ├── models/ShopRecord.ts     # Mongoose schema ← BEING CHANGED
│   ├── mongodb.ts
│   ├── utils.ts
│   └── validation.ts
├── .env.local.example
└── package.json
```

3. Current MongoDB Schema (ShopRecord)

Each extracted row from a sheet is one ShopRecord document. The image is currently stored as base64 INSIDE each ShopRecord. This means if a sheet has 3 rows, the same large image is duplicated 3 times in MongoDB. This is the problem being solved.

```
ShopRecord {
  status: "Needs Review" | "Approved"
  original_image_base64: string // <-- PROBLEM: duplicated per row
  date: string | null
  shift: number | null // 1, 2, or 3
  emp_no: string | null
  opn_code: string | null
  machine_no: string | null
  work_order_no: string | null
  qty_produced: number | null
  time_taken_hrs: number | null
  confidence_scores: {
    date, shift, emp_no, opn_code,
    machine_no, work_order_no, qty_produced, time_taken_hrs // each 0-100
  }
  validation_errors: string[]
  createdAt: Date
  updatedAt: Date
}
```

4. What Needs to Change — Full Specification

4a. New MongoDB Architecture

Create a second Mongoose model called **UploadSession**. This stores the image ONCE per upload. All ShopRecord rows from that upload reference the UploadSession by ID.

```
// NEW MODEL: lib/models/UploadSession.ts
UploadSession {
  original_image_base64: string // stored once per upload
  mime_type: string // e.g. "image/jpeg"
  row_count: number // how many rows were extracted
  createdAt: Date
  updatedAt: Date
}

// UPDATED MODEL: lib/models/ShopRecord.ts
ShopRecord {
  upload_session_id: ObjectId // reference to UploadSession <-- NEW
  status: "Needs Review" | "Approved"
  // original_image_base64 REMOVED from here
  date: string | null
  shift: number | null
  emp_no: string | null
  opn_code: string | null
  machine_no: string | null
  work_order_no: string | null
  qty_produced: number | null
  time_taken_hrs: number | null
  confidence_scores: { ... } // unchanged
  validation_errors: string[] // unchanged
  createdAt: Date
  updatedAt: Date
}
```

4b. Updated Upload Route

In **app/api/upload/route.ts**, after Gemini/Mistral extraction:

1. Save the image ONCE as a new UploadSession document, storing base64 + mimeType + row_count.

2. For each extracted row, create a ShopRecord with upload_session_id pointing to the UploadSession. Do NOT store the image in ShopRecord.

3. Return shape stays the same: **{ records: ShopRecord[], provider: string }**. Optionally also return the session ID.

4c. Updated Documents API

app/api/documents/route.ts — The GET list endpoint currently excludes the image. Now it must also populate/join the UploadSession so the response includes upload_session_id and the session's createdAt and row_count (for grouping in the UI). Use Mongoose populate or a manual lookup.

app/api/documents/[id]/route.ts — The GET single record endpoint must now also return the image. Populate the upload_session_id field so the full base64 image is available for the edit screen. PUT and DELETE stay unchanged.

4d. New Sessions API

Create **app/api/sessions/route.ts** — GET endpoint that returns all UploadSessions sorted by createdAt desc, each with their row_count and a summary of their rows (statuses, any validation errors). This powers the grouped History view.

```
// Response shape for GET /api/sessions
[
  {
    _id: string,
    createdAt: Date,
    row_count: number,
    mime_type: string,
    rows: [
      {
        _id: string,
        status: "Needs Review" | "Approved",
        date: string | null,
        shift: number | null,
        emp_no: string | null,
        machine_no: string | null,
        work_order_no: string | null,
        qty_produced: number | null,
        time_taken_hrs: number | null,
        validation_errors: string[]
      }
    ]
  }
]
```

4e. Completely Replaced History Page

app/history/page.tsx — Completely replace the current flat table view with the new grouped card view. Behaviour:

Default view (collapsed cards): Each UploadSession is one card. Card shows: upload date/time, number of rows, and a row status summary (e.g. '2 Approved, 1 Needs Review'). Cards are sorted newest first.

Search/filter bar: Keep the existing search (by work order, emp no, machine) and status filter. Filtering operates across all sessions — sessions with no matching rows are hidden entirely.

Expanding a card: Clicking a card expands it inline to show the split-screen edit view — image on the left (fetched from the UploadSession), row navigator tabs on top, editable form on the right. This is the EXACT same layout as app/page.tsx (Upload and Review page). Reuse the same components/logic.

Saving from History: Uses the existing PUT /api/documents/[id] endpoint. After saving, the card's status summary updates.

5. Files You Must Upload to the New Claude Session

Upload these exact files alongside this PDF:

File Path	Why Needed
lib/models/ShopRecord.ts	Claude will modify this to remove image field and add upload_session_id
lib/validation.ts	Claude needs to check it requires no changes
app/api/upload/route.ts	Claude will modify the save logic
app/api/documents/route.ts	Claude will update to populate session data
app/api/documents/[id]/route.ts	Claude will update GET to return image via populate
app/history/page.tsx	Claude will fully replace this file
app/page.tsx	Claude will extract reusable review components from this

6. Exact Prompt to Send to the New Claude Session

Send this as your first message alongside this PDF and all the files listed in Section 5:

I have a Next.js 15 + MongoDB + Mongoose project. I am uploading a handoff PDF that explains everything, plus the current source files listed in Section 5 of the PDF.

Please implement all changes described in Section 4 of the PDF exactly:

- Create lib/models/UploadSession.ts (new model)
- Update lib/models/ShopRecord.ts (remove image, add upload_session_id)
- Update app/api/upload/route.ts (save image to UploadSession once)
- Update app/api/documents/route.ts (populate session data)
- Update app/api/documents/[id]/route.ts (return image via populate on GET)
- Create app/api/sessions/route.ts (new grouped sessions endpoint)
- Replace app/history/page.tsx (new grouped card UI with inline split-screen edit)

Give me each file completely, one at a time. Do not summarise or truncate any file.

7. Important Notes for Claude

The Upload and Review page (app/page.tsx) must NOT be changed. The History edit view replicates its split-screen layout but is self-contained inside history/page.tsx using the same logic pattern.

The existing shadcn/ui components (button, card, badge, input, label, alert) are already in components/ui/. Do not recreate them. Import from @/components/ui/.

The Gemini/Mistral fallback logic in upload/route.ts must be preserved exactly — only the save logic at the bottom changes.

Keep the existing validation logic in lib/validation.ts completely unchanged.

All existing API routes (analytics, documents PUT/DELETE) remain unchanged unless explicitly listed in Section 4.

The app uses Next.js App Router. All API routes use NextRequest/NextResponse. No pages router.

TypeScript strict mode is on. All new code must be fully typed with no 'any' unless unavoidable.

End of handoff document.